Applied 2: Presentation Script

793 spoken words. Target: 6:40, with 20 seconds available before seven minutes. At an assumed 130 words per minute, speech takes about 6.1 minutes, leaving time for pauses and slide changes. This is a planning estimate, not a measured rehearsal.

Suggested split: Savin presents slides 1–6; Shamle presents slides 7–11. Stop at the conclusion; the appendix is not part of the script.

Savin Vindiv De Alwis (35221631) — sdea0018@student.monash.edu
Shamle Thilaksiri (35512075) — wthi0003@student.monash.edu

Italic cues are directions; speak only the main paragraphs.

Slide 1 — Introduction

Savin • 8 seconds • Finish by 0:08

Start on the cover.

We’re Savin and Shamle. We’ll cover GPU rotation and responsible HPC scaling.

Slide 2 — Moving the image to the GPU

Savin • 25 seconds • Finish by 0:33

Follow the arrows from storage to the final host buffer.

The CPU decodes the image into host RAM and allocates GPU buffers. DMA copies the pixels over PCI Express; the GPU rotates them, and completed output returns for saving. Pinned RAM helps DMA; pageable RAM may need staging. Device bandwidth and transfer bandwidth are different. We wait before saving or reusing buffers.

Slide 3 — How the rotation works

Savin • 30 seconds • Finish by 1:03

Point to the inverse-mapping equations, then the output-to-input diagram. Do not read the equations aloud.

Each thread owns one output pixel. We rotate backwards about the image centre, with the angle in radians, to find the nearest source pixel and copy its RGB values. Outside the image, we write black. Independent writes need no atomics or barrier. Positive angles are counterclockwise with our y-down coordinates. Gathering avoids holes, but a fixed canvas can crop corners and nearest-neighbour sampling can alias.

Task 1: CUDA and GPUDirect Storage

Slide 4 — CUDA threads, blocks and grids

Savin • 42 seconds • Finish by 1:45

Point to the grid/block launch, then the scheduling diagram. Mention the appendix examples without opening them.

The CPU launches a kernel with grid and block settings. Sixteen-by-sixteen blocks contain two hundred and fifty-six threads, or eight warps. This sixty-four-by-sixty-four grid describes about a million threads. They don’t all run at once: finite streaming multiprocessors schedule blocks in waves. SIMT means a warp follows common instructions, with thirty-two threads per warp. Threads compute pixel indices and guard image edges; other sizes need ceiling division. Launches are asynchronous, so check errors and completion. Full rotation, brightness, block-shape and stream examples are in the appendix.

Slide 5 — Expected speedup

Savin • 45 seconds • Finish by 2:30

Emphasize the full workload, rather than only the kernel.

Many GPU execution units and high memory bandwidth process independent pixels together. Coalescing combines nearby addresses into fewer transactions; rotated reads can limit bandwidth. Thirty-two-by-eight blocks keep a warp on one row, so we’d profile them against sixteen-by-sixteen. Resident warps hide latency, but registers and divergent branches limit execution. Large images amortize setup; keeping pixels on the GPU avoids repeat copies. Streams overlap independent images with separate pinned buffers and supporting hardware, but need more memory. Compare identical rotation against serial and multithreaded CPUs, timing setup, transfers and completed work. Serial work bounds speedup; no measured result is claimed.

Slide 6 — Where GPUDirect Storage helps

Savin • 30 seconds • Finish by 3:00

Compare the two storage paths. Hand over after the final sentence.

GPUDirect Storage uses storage-side DMA to bypass host staging; the CPU still issues cuFile requests. It doesn’t decode images or accelerate the rotation kernel. CPU-decoded or already-in-RAM pixels gain little from a storage bypass. GPU-ready, storage-bound batches may benefit, but GPU, filesystem, drivers and topology must support the direct path. Fallback may still stage through RAM. We’d measure completed batch time and CPU overhead. Shamle will cover responsible scaling.

Task 2: responsible HPC scaling

Slide 7 — Energy and environmental costs

Shamle • 60 seconds • Finish by 4:00

Point to the sector trend, then the job-level accounting. Mention the supporting charts and calculation in the appendix.

In data-parallel training, GPUs process different batches, and gradient all-reduce synchronizes updates. Communication and slow workers limit scaling. The IEA projects near-doubling of data-centre electricity by 2030, but that's whole-sector context, not one model's footprint. For a job, energy depends on power over time; carbon also depends on local carbon intensity. Faster can still use more energy. Count CPUs, networking, storage, cooling, tuning and failed runs, plus manufacturing impacts. Green AI measures efficiency per useful result; Li's water research adds local resource pressure. Lower carbon alone can miss water costs. The appendix has the detailed calculation, growth charts and research comparison.

Slide 8 — Security, governance and fairness

Shamle • 60 seconds • Finish by 5:00

Explain the security side, then move to the fairness side.

Distributed replicas, gradients and checkpoints add assets to protect. Zhu's study shows input reconstruction from exposed gradients in studied settings; it doesn't prove every collective leaks. Use least privilege, isolation and protected transport and storage. Named owners check data provenance, permissions, replica retention and deletion, and dual-use risks. Fairness needs different controls. For a multilingual LLM, reserve compute to compare harmful-output rates across languages, examine uncertainty, improve underrepresented-language coverage and retest. Gallegos's survey explains why metrics and datasets matter; more GPUs can't fix unrepresentative data. Security protects intermediates; fairness tests outcomes. The full risk register and evaluation details are in the appendix.

Task 2: responsible HPC scaling

Slide 9 — Who gets access to compute?

Shamle • 35 seconds • Finish by 5:35

Keep the emphasis on participation, allocation and the trade-off.

Compute access affects who can train, reproduce and shape AI research. Ahmed and Wahed study historical participation; Strubell examines workload costs. Their scopes differ, but both reveal barriers. NAIRR's US reach doesn't prove global equality. Publish GPU-hour rules, reserve smaller-institution and replication capacity, offer training and appeals, and audit awards and waiting times with privacy safeguards. Reservations can reduce utilization or delay urgent jobs; prevent starvation.

Slide 10 — Our responsible-use framework

Shamle • 45 seconds • Finish by 6:20

Follow Map, Measure and Manage. Indicate that Govern applies throughout.

OECD supplies rights and accountability; NIST structures risk decisions. Govern assigns ownership throughout. Map defines the use, affected people and alternatives. Measure checks quality, energy, security and group harms. Manage approves, defers, reduces scale or redesigns. Green AI tells us to measure efficiency; Radovanovic's scheduling research shows how forecasts can shift flexible work. Measurement and action serve different needs. Set an energy budget and shift jobs only within deadlines, capacity, water limits and fair waiting commitments. Record urgent or forecast exceptions and review outcomes. This is an iterative proposal, not a certification; the detailed policy is in the appendix.

Slide 11 — Conclusion

Shamle • 20 seconds • Finish by 6:40

Finish here. Do not continue into the appendix.

CUDA suits independent pixels, but performance still needs measurement. Responsible scaling needs useful outcomes, protected data and fair access. Next, compile and validate the rotation, then measure a real cluster job and review its impacts. Our code examples are uncompiled and unbenchmarked.